

01 — WORKSHOP

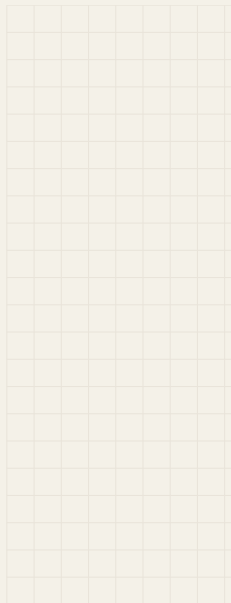
Introduction to GNU Radio

Build a radio in software: flowgraphs, filters, receivers and custom blocks.

Derek Kozel

September 21, 2026

- 01 Signals, sampling and SDR
- 02 Your first flowgraph
- 03 Noise, filters and tuning
- 04 Modulation and receivers
- 05 GNU Radio from Python
- 06 Under the hood



How this session runs

Watch, build, play

Each part is a short explanation, then you build something, then you change it until it misbehaves.

Everything runs in simulation. If you have an SDR, one section puts it in front of the same flowgraphs.

Files to download

<https://github.com/dkozel/grcon26-workshop>

- `flowgraphs/` — the finished version of every exercise
- `python/` — the scripts for Part 5

Fall behind? Open the finished flowgraph and carry on.

Before we start

GNU Radio 3.10 or newer, and a working soundcard.

- `gnuradio-config-info --version`
- `gnuradio-companion` opens
- `python3 python/01_hello_flowgraph.py`

That last one should print:

Collected 32000 complex samples

Strongest frequency: 1000.0 Hz

If it does not work

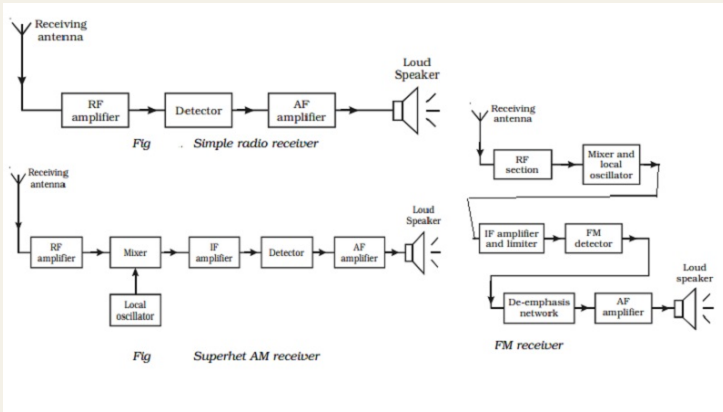
Sit next to someone whose does. Ask for help.

01 —

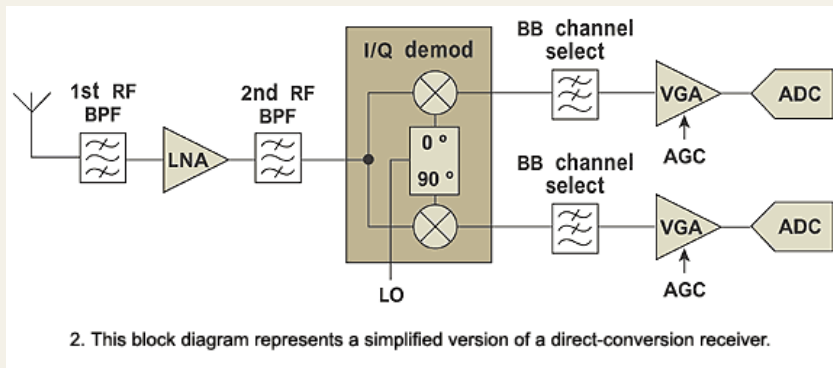
Signals, sampling and SDR

What a radio does, and which half of it we are moving into software.

Traditional radio architectures

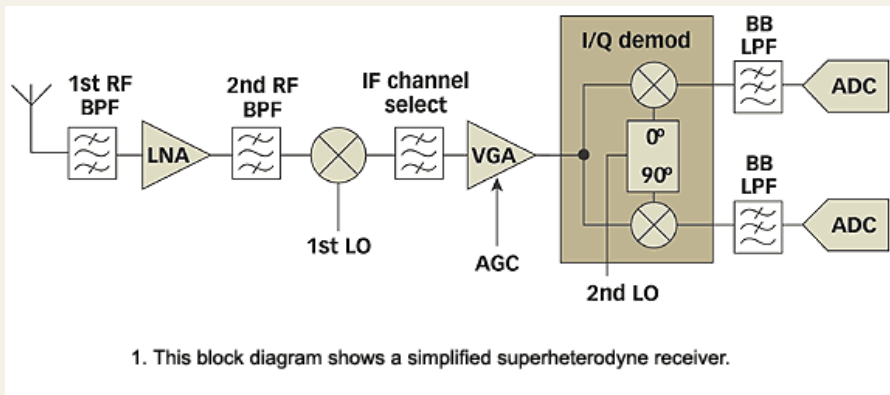


What SDR changes: direct conversion



Tune the oscillator to the signal, shift it to baseband, digitise, and do *everything else* in software.

The superheterodyne



Mix to a fixed intermediate frequency, filter there, mix to baseband, and digitize

Why bother

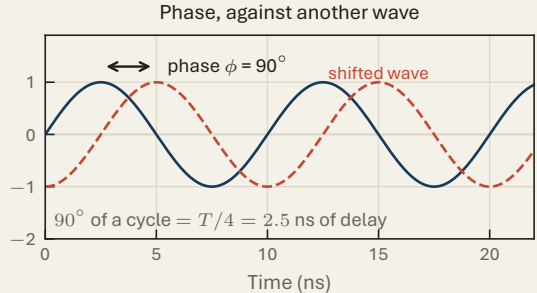
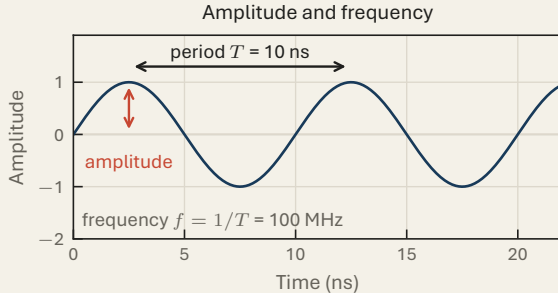
- **Flexibility** — same hardware, different program: FM radio, aircraft, satellites
- **Upgradability** — a protocol change is a software change
- **Visibility** — you can see the signal at every stage

Hardware never disappears

Antennas, filters, amplifiers and the converters are still physical, and their limits still apply.

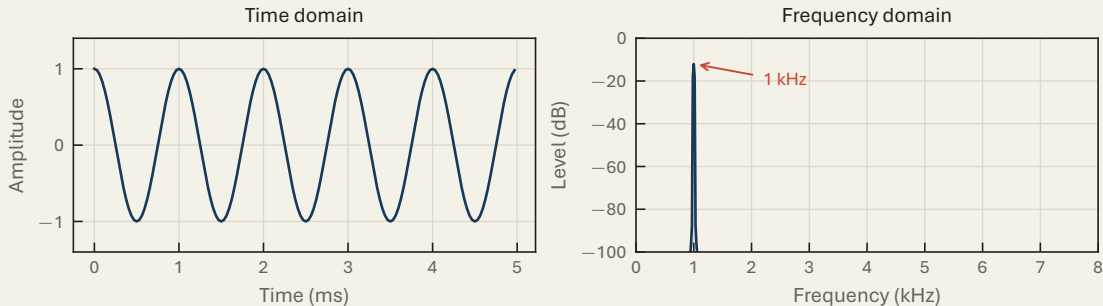
The term is usually credited to Joe Mitola, 1991.

A wave is three numbers



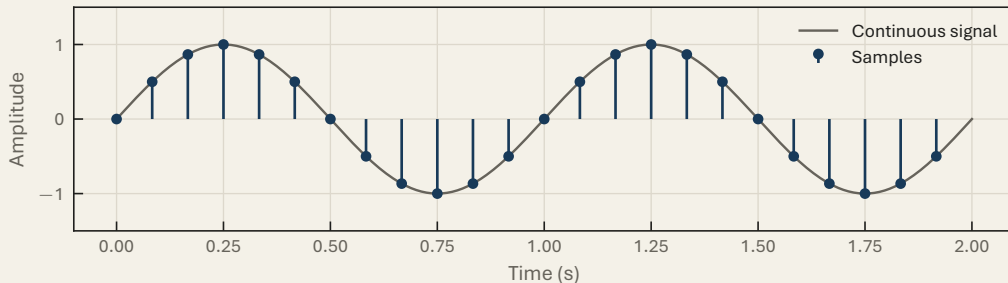
Frequency (cycles per second) • **amplitude** (how strong) • **phase** (relative to another wave). Wavelength follows: $\lambda = c/f \approx 300/f_{\text{MHz}}$, 3 m at 100 MHz.

Two views of the same signal



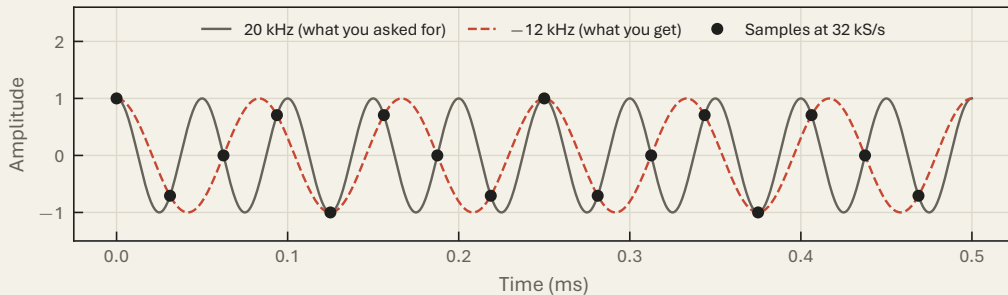
Any signal is a sum of sine waves; the frequency view shows how much energy sits at each one.

Sampling



Software processes a list of numbers, measured at a steady rate: the **sample rate**. It is the most important number in any flowgraph.

Aliasing

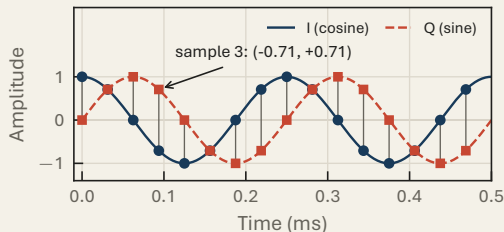


At 32 kS/s a 20 kHz tone and a -12 kHz tone produce *identical* samples. Nothing downstream can tell them apart.

Complex samples (IQ)

Each sample is a pair: **I** and **Q**. That falls straight out of the direct conversion receiver, which multiplies by both a cosine and a sine.

- Real signal: bandwidth = rate / 2, spectrum mirrored
- Complex signal: bandwidth = *the whole sample rate*
- "Negative frequency" = below where the radio is tuned



In GNU Radio

A complex sample is two 32-bit floats. Amplitudes are conventionally kept between -1.0 and $+1.0$.

What GNU Radio is

- **Blocks** do one job each
- Wired into a **flowgraph**; samples stream through buffers
- Output only = **source**; Input only = **sink**
- C++ engine, Python wrapper
- Every block runs in **its own thread**

Free and open source, started by Eric Blossom in 2001.

GRC is not an interpreter

GNU Radio Companion *generates a Python program* from your flowgraph. You can read it, keep it and edit it — Part 5.

Data types have colours

Colour	Type	Typical use
Blue	Complex float32	IQ samples
Orange	Float32	audio, magnitudes
Teal	Int32	counters
Yellow	Short	compact IQ
Magenta	Byte	bits and bytes
Grey	Message	asynchronous data

GRC refuses to connect mismatched ports.

- *Help* → *Types* shows this table
- Select a block, press $\uparrow / \downarrow$ to change type
- Converter blocks bridge the rest

02 —

Your first flowgraph

Build a signal generator, then break it.

The GRC window

The screenshot displays the GNU Radio Companion (GRC) interface. At the top is a menu bar (File, Edit, View, Run, Tools, Help) and a toolbar with icons for file operations, running, and zooming. A red box highlights the menu and toolbar, with the text "Menu and toolbar (run the flowgraph with the ► button)".

The main canvas area contains a flowgraph. It starts with a "Signal Source" block (Sample Rate: 32k, Waveform: Cosine, Frequency: 1k, Amplitude: 1, Offset: 0, Initial Phase (Radians): 0). This is connected to a "Throttle" block (Sample Rate: 32k, Limit: None). The output of the throttle is split into two paths: one to a "QT GUI Range" block (ID: freq, Label: Frequency (Hz), Default Value: 1k, Start: 0, Stop: 10k, Step: 100) and another to a "QT GUI Time Sink" block (Name: Time domain, Number of Points: 256, Sample Rate: 32k, Autoscale: No). A red box labeled "Canvas: your flowgraph lives here" points to this central area.

On the right side, there is a "Block library: Ctrl+F to search" section. It lists various block categories with expandable arrows: Core, Boolean Operators, Byte Operators, Channelizers, Channel Models, Coding, Control Port, Debug Tools, Deprecated, Digital Television, Equalizers, Error Coding, File Operators, Filters, Fourier Analysis, GUI Widgets, Impairment Models, Instrumentation, IQ Correction, Level Controllers, Math Operators, Measurement Tools, Message Tools, Misc, Modulators, Networking Tools, OFDM, Packet Operators, PDU Tools, Peak Detectors, Resamplers, Stream Operators, and Stream Tag Tools.

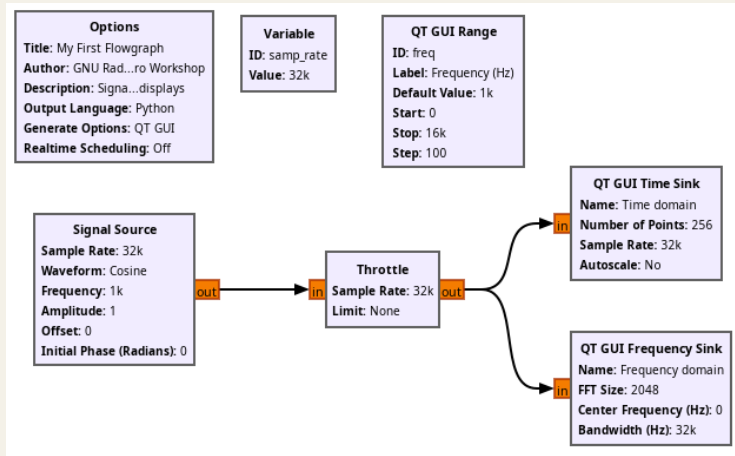
At the bottom left, the "Console: errors and warnings" section shows the following text:

```
>>> Welcome to GNU Radio Companion
Loading: "/01_first_flowgraph.grc"
>>> Done
```

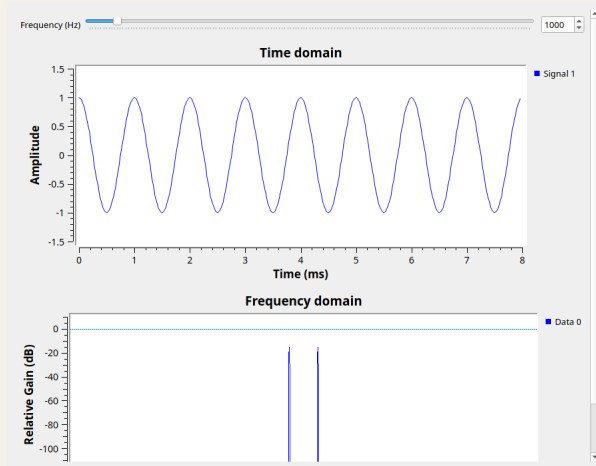
At the bottom right, the "Variable editor" section shows a table of variables:

ID	Value
Imports	
Variables	
freq	1000
samp_rate	32000

The flowgraph



What you should see



Exercise 2.1 — a signal generator

- 01** *File* → *New*, QT GUI. Save it.
- 02** **Options:** ID = `first_flowgraph`
- 03** **Variable:** `samp_rate = 32000`
- 04** **Ctrl+F** → **Signal Source:** Float, `samp_rate`, Cosine, 1000 Hz, amplitude 1
- 05** **Throttle:** Float, `samp_rate`
- 06** **QT GUI Time Sink:** Float, 256 points
- 07** **QT GUI Frequency Sink:** Float, bandwidth `samp_rate`
- 08** Wire source → throttle → *both* sinks
- 09** Press ► (Run)

Red block title?

A parameter is invalid. Read the console.

Why two spikes?

A *real* cosine cannot tell $+1$ kHz from -1 kHz, so its spectrum is always symmetric about zero.

A complex tone shows one line. We will put them side by side in Exercise 2.3.

Worth knowing

Each of the two real lines carries half the amplitude, so it sits exactly **6 dB** below the single line of a complex tone of the same size.

Decibels

$$\text{dB} = 10 \log_{10} \frac{P_1}{P_0} = 20 \log_{10} \frac{A_1}{A_0}$$

Radio spans a billion to one. Logarithms make that readable, and turn the gains along a chain into a sum.

dB	Power	Amplitude
+3	$\times 2$	$\times 1.41$
+6	$\times 4$	$\times 2$
+10	$\times 10$	$\times 3.16$
+20	$\times 100$	$\times 10$
-6	$\div 4$	$\div 2$
-60	$\div 10^6$	$\div 1000$

Relative to what? Full scale: a sample of amplitude 1.0 is 0 dB (dBFS). GNU Radio cannot know what your antenna and amplifiers did, so it never shows you dBm.

Reading the frequency display

It collects FFT Size samples, transforms them, and plots the energy in each **bin**.

$$\text{bin width} = \frac{\text{sample rate}}{\text{FFT size}} = \frac{32000}{2048} \approx 15.6 \text{ Hz}$$

A bigger FFT resolves finer detail but needs more samples: frequency resolution versus time resolution, permanently.

Leakage and windows

A tone rarely lands exactly on a bin centre, so it smears into its neighbours — the skirts on your spike. The Window parameter tapers each block of samples to push those down.

Same idea returns in Part 3, designing filters.

The Throttle block

GNU Radio runs as fast as it can. Something must set the pace, and exactly one thing should.

- Simulation only → add **one Throttle**
- SDR or **Audio** hardware present → **no** throttle

Symptoms

No throttle in a simulation: CPU cores at 100%.

Throttle plus hardware: two clocks fighting, stuttering audio.

GRC warns about both in the console.

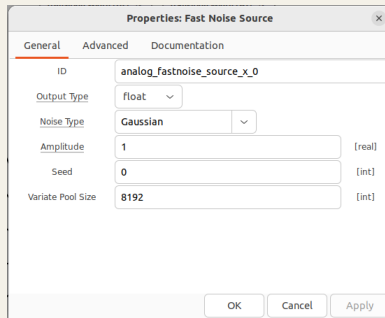
Variables and sliders

A **QT GUI Range** is a variable with a slider. Any parameter can refer to it by name, and moving the slider updates all of them while the flowgraph runs.

Parameters are *Python expressions*: `samp_rate/2`, `int(10**6)`, `firdes.low_pass(1.0, samp_rate, 5e3, 1e3)`.

Only underlined parameters change at runtime

The rest are read once, when the blocks are built.
Stop, change, restart.

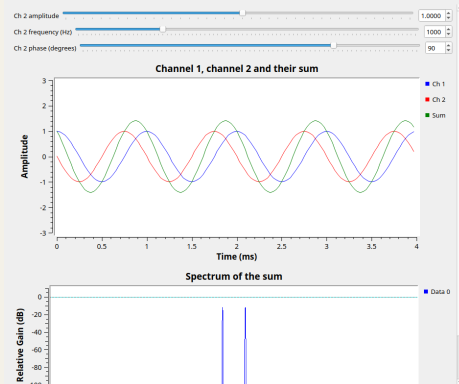


Exercise 2.2 — amplitude, frequency, phase

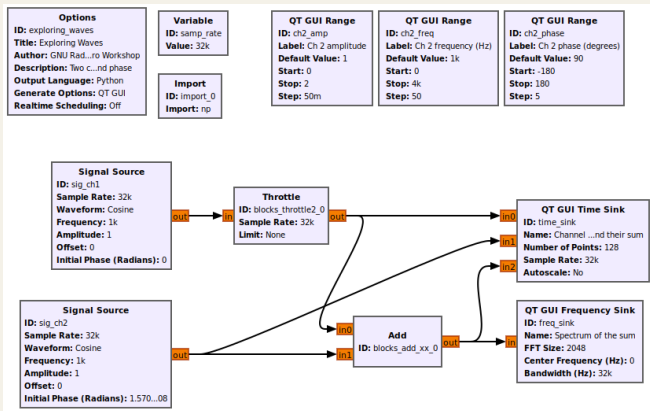
02_exploring_waves.grc

- Phase 0: the sum is twice as tall
- Phase 180: they cancel *completely*
- Channel 2 at 1010 Hz: a 10 Hz **beat**

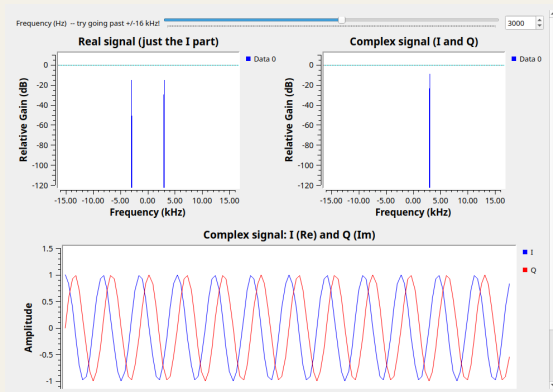
The **Import** block brings in `numpy` so the slider can work in degrees.



Exercise 2.2 — amplitude, frequency, phase

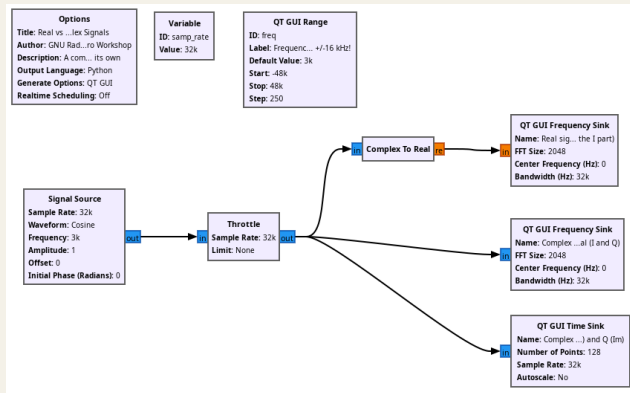


Exercise 2.3 — real versus complex

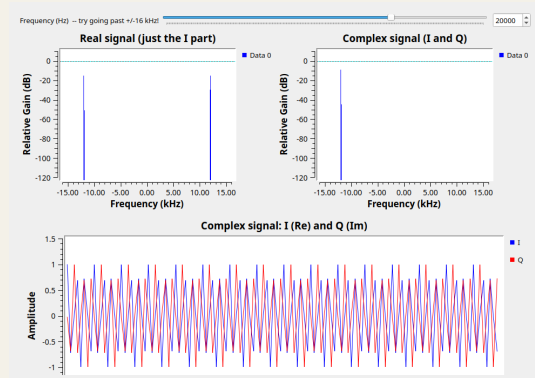


One complex source; **Complex To Real** keeps only I. Two lines versus one, 6 dB apart.

Exercise 2.3 — real versus complex



Now break Nyquist



Slider past 16 kHz: at 20 kHz the spike reappears at -12 kHz. **This is why every rate change is preceded by a filter.**

03 —

Noise, filters and tuning

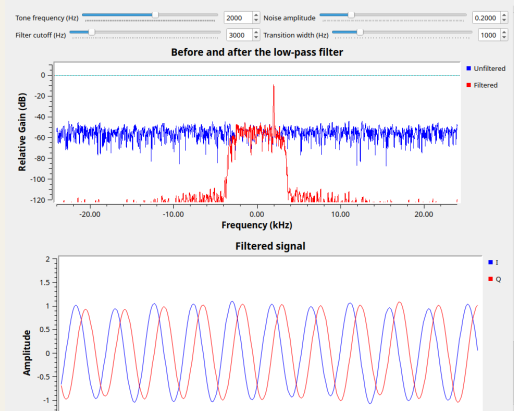
Clean a signal up, then pick one station out of a band.

Exercise 3.1 — a tone in noise

04_noise_and_filtering.grc

A complex tone, plus Gaussian noise, through a **Low Pass Filter**. Blue is before, red is after.

- Raise the noise — observe changes in time and frequency views
- Move the tone outside the passband
- Narrow the transition width



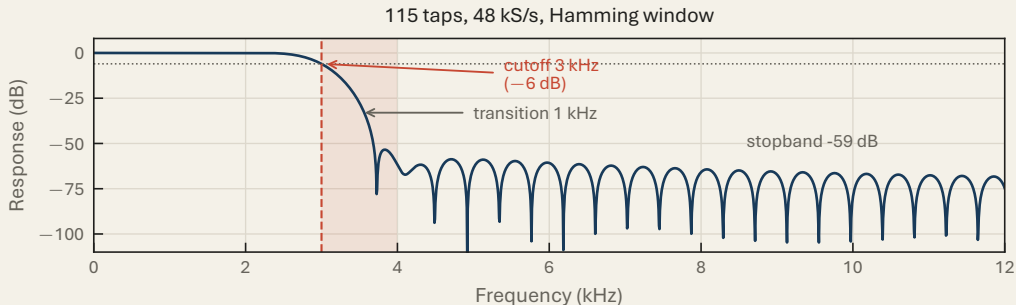
The parameters that matter

Parameter	Meaning
Cutoff Freq	the -6 dB point
Transition Width	how fast it gets to the stopband
Gain	passband gain, usually 1
Window	ripple versus length
Decimation	drop the output rate by N

A common surprise

With real taps and a complex input the passband runs from $-\text{cutoff}$ to $+\text{cutoff}$ — 6 kHz wide, not 3.

What "cutoff" actually means



0 dB at 2 kHz, -6 dB at the 3 kHz cutoff, -27 dB at 3.5 kHz, below -58 dB past 4 kHz.

Sharper costs taps

Transition width	Taps
2000 Hz	57
1000 Hz	115
500 Hz	231
250 Hz	463

Halve the transition width, double the taps, double the CPU.

Sharp filters are not free.

A little theory: linear and time invariant

Linear: double the input, double the output; the response to a sum is the sum of the responses.

Time invariant: same signal tomorrow, same answer, just later.

Almost every block so far is **LTI**. The clipping block you write in Part 5 deliberately is not — which is why it invents new frequencies.

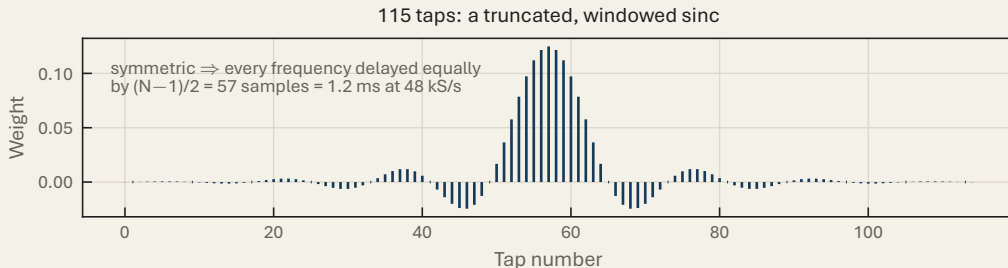
Why it matters

An LTI system is completely described by its **impulse response**, and its output is the input **convolved** with it:

$$y[n] = \sum_k h[k] x[n - k]$$

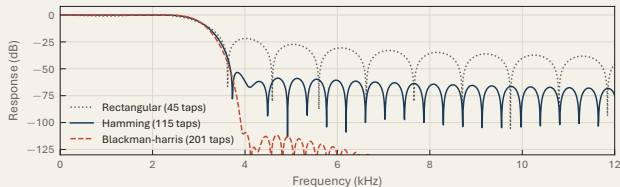
Convolution in time is multiplication in frequency — which is why we may draw a filter as a *shape*.

A FIR filter's taps *are* its impulse response



`firdes.low_pass(1.0, samp_rate, cutoff, transition)` returns this list of numbers. 115 taps = 115 multiply-accumulates per output sample.

Why windows exist



The ideal filter's impulse response is an infinite sinc. Truncating it bluntly gives ripple; a **window** tapers the truncation.

Window	Taps	Stopband
Rectangular	45	-22 dB
Hamming	115	-59 dB
Blackman-h.	201	-109 dB

Same cutoff, same transition. Depth costs length.

Linear phase, and the delay a filter costs

`firdec` makes symmetric taps, so every frequency is delayed by the same $(N - 1)/2$ samples.

115 taps at 48 kS/s = 57 samples = **1.2 ms**.

Constant delay means the waveform keeps its shape — which matters enormously for digital modulation.

What about IIR?

Feedback makes a few coefficients as selective as a hundred taps, at the price of non-linear phase and possible instability.

The Alpha on **Power Squelch** in Part 4 is exactly this: a one-pole IIR averager.

Mixing: multiplication moves signals

Multiply by a complex sinusoid at f_0 and the whole spectrum slides by f_0 . That is tuning.

- Negative f_0 brings a station *down* to 0 Hz
- A complex multiply makes only the shifted copy
- Two *real* signals multiplied give sum *and* difference — the extra copy analogue designers must filter away

Decimation

Once a signal is at 0 Hz and filtered to its own bandwidth, most of your sample rate is empty spectrum. Keep one sample in N and everything downstream gets cheaper.

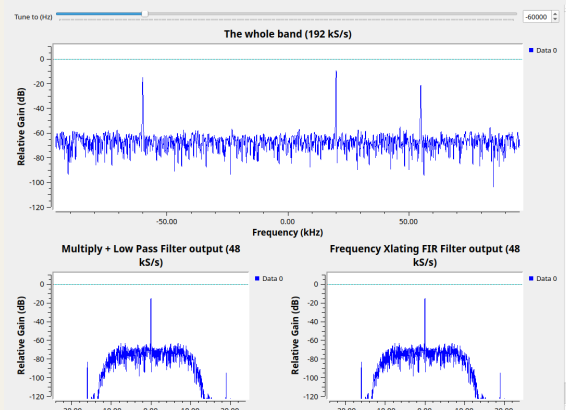
Exercise 3.2 — tune a station out of a band

05_tuning_and_decimation.grc

Three stations in 192 kHz, and the same job done twice:

- **Multiply** by an oscillator at $-tune$, then **Low Pass Filter** with decimation 4
- One **Frequency Xlating FIR Filter**

Identical outputs at 48 kS/s. The second is what you will use.

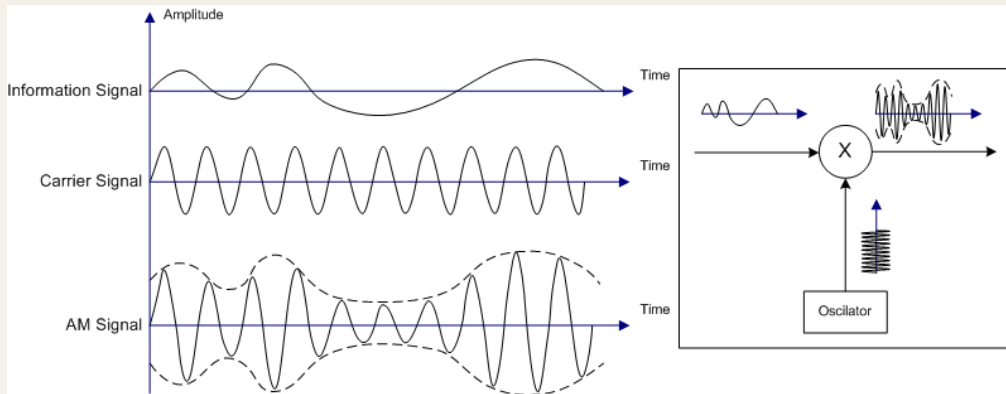


04 —

Modulation and receivers

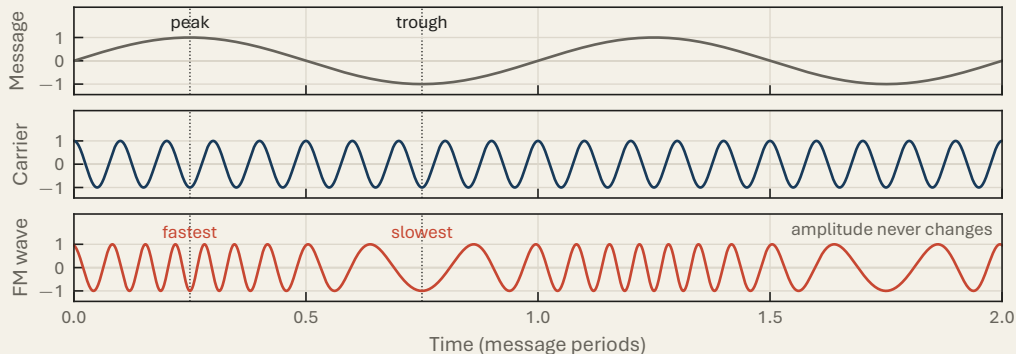
Put information on a carrier, then get it back.

Amplitude modulation



Vary the height of the carrier with the message.

Frequency modulation



Keep the amplitude constant, vary the rate instead. Noise mostly disturbs amplitude — which is why FM sounds clean and AM hisses.

Exercise 4.1 — an AM transmitter and receiver

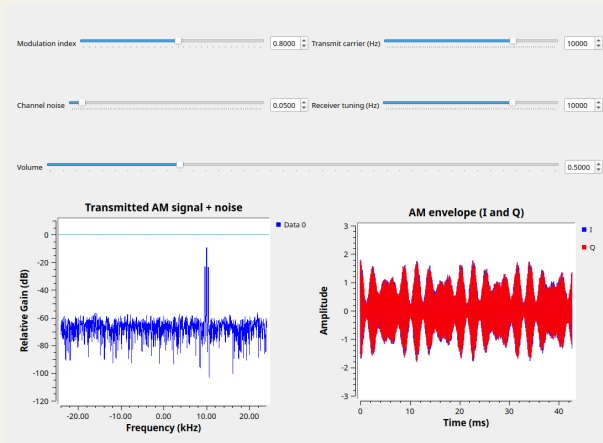
Transmitter

- 01** Two tones, 350 + 440 Hz: a dial tone
- 02** $\times \text{mod_index}$, + 1
the message now swings around 1.0
- 03** **Float To Complex**, then **Multiply** by a 10 kHz carrier
- 04** Add noise: a channel
No **Throttle**: the soundcard is the clock.

Receiver

- 05** **Frequency Xlating FIR Filter** tunes and filters to ± 5 kHz
- 06** **Complex to Mag** — the envelope detector.
AM demodulation really is one block
- 07** **DC Blocker**, volume, **Audio Sink**

What you should see — and hear

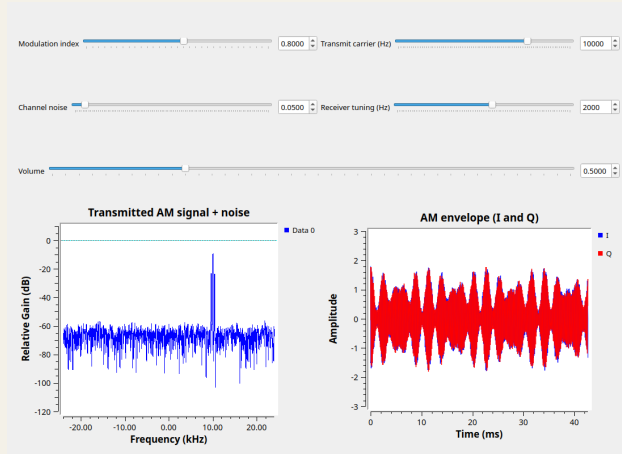


Things to try

- **Modulation index above 1.0** —
 $1 + m \cdot \text{message}$ goes negative, the envelope folds, the audio distorts. Over-modulation.
- **Raise the noise** until the dial tone is buried.
AM has no protection against amplitude noise at all.
- **Mistune by 2 kHz** — and it still works. A *magnitude* does not care about a frequency offset.
- **Mistune by 8 kHz** — now the signal is outside the receiver's filter, and it goes silent.

Measured at the soundcard: 0.20 RMS tuned, 0.005 mistuned by 8 kHz.

Mistuned



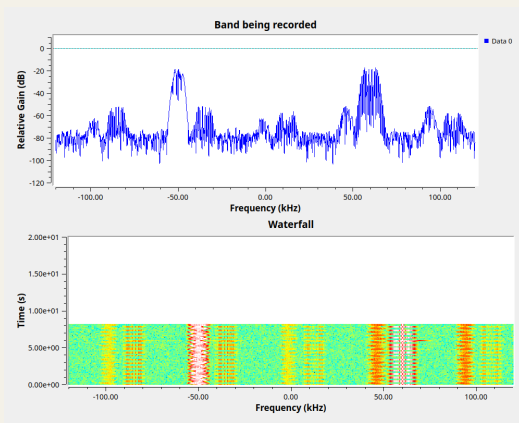
Exercise 4.2 — record a band

07_fm_band_recorder.grc

Two narrowband FM stations, 240 kS/s:

- -50 kHz: the dial tone
- +60 kHz: 800 Hz beeps

Run it for **15 seconds**, then close it. You now have an IQ recording to receive from.



The file you just made

A `.cfile` is raw interleaved 32-bit floats: I, Q, I, Q, with **no header**.

At 240 kS/s that is $240000 \times 8 = \mathbf{1.92 \text{ MB per second}}$ — about 29 MB for 15 seconds.

Nothing records the rate

Sample rate and centre frequency are not in the file, which is why everyone ends up putting them in the filename.

Choosing Short or Byte output halves or quarters the size, at the cost of dynamic range.

Exercise 4.3 — the receiver

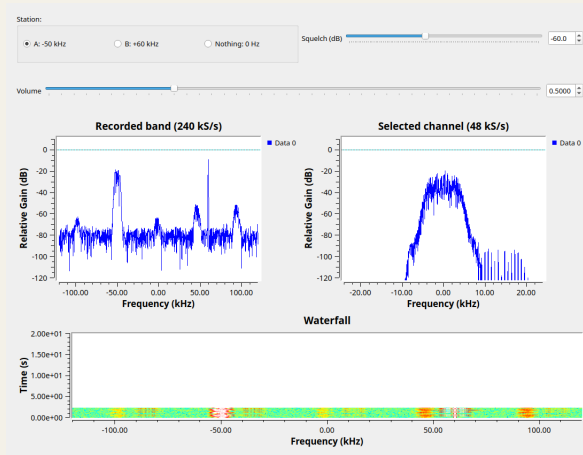
Stage	Block
Source	File Source , Repeat
Tune, filter, decimate	Freq Xlating FIR , $\div 5$
Mute the noise	Power Squelch
Demodulate	NBFM Receive
Volume	Multiply Const
Sink	Audio Sink

$240 \text{ kS/s} \div 5 = 48 \text{ kS/s}$, the audio rate. Again no throttle.

This shape is every receiver

Get samples, pick a channel, drop the rate, demodulate, listen. Everything else is a variation.

Pick a station



Squelch

Tune to the empty channel: loud hiss, because an FM demodulator with no signal produces noise at full volume.

Raise **Squelch** to about -30 dB. Silence — and the stations still work.

Two details

The threshold is in dB *relative to full scale*, not dBm.

Gate: No outputs zeros (silence). Gate: Yes stops producing samples and starves the soundcard.

The two-clock problem

A real receiver has two clocks: the SDR's and the soundcard's. They are never exactly equal.

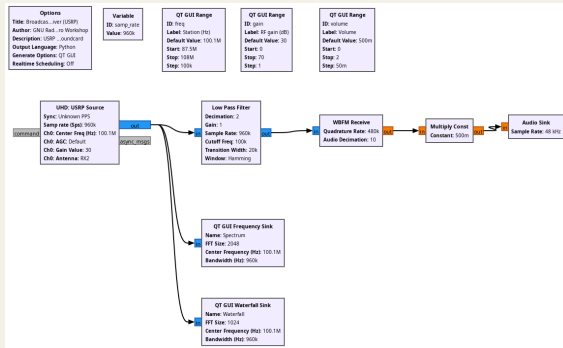
- Source slightly **fast** → samples pile up, latency grows
- Source slightly **slow** → the soundcard underruns: clicks and gaps

Fixes, best first: one clock for both ends; bigger buffers; reduce CPU load; resample against the measured difference.

Letter	Meaning
aU	audio underrun
U	transmit underrun
O	receive overrun
S	sequence error

Printed without a newline, forever, while it happens.

With real hardware



Broadcast FM: 960 kS/s in, filter and decimate to 480, **WBFM Receive** to 48 kHz audio.

- Swap the source block for your device; the rest is unchanged
- Start gain half way
- Check the antenna port
- Not every sample rate exists
- Transmitting is regulated

05 —

GNU Radio from Python

The same flowgraphs, as code you
can test. *30 minutes*

GRC writes Python

```
class first_flowgraph(gr.top_block, Qt.QWidget):
    def __init__(self):
        gr.top_block.__init__(self, "My First Flowgraph")
        self.samp_rate = samp_rate = 32000
        self.freq = freq = 1000

        self.analog_sig_source_x_0 = analog.sig_source_f(
            samp_rate, analog.GR_COS_WAVE, freq, 1, 0, 0)

        self.connect((self.analog_sig_source_x_0, 0),
                     (self.blocks_throttle2_0, 0))

    def set_freq(self, freq):
        self.freq = freq
        self.analog_sig_source_x_0.set_frequency(self.freq)
```

- A flowgraph is a class derived from `gr.top_block`
- Every variable gets a **setter** — that is all a slider does
- Regenerated on every run: never edit it in place

Exercise 5.1 — a flowgraph in pure Python

python/01_hello_flowgraph.py

Signal Source -> Add (noise) -> Head -> Vector Sink

Collected 32000 complex samples

Strongest frequency: 1000.0 Hz

Average power: 0.04 dB

- **Head** passes N samples then stops the flowgraph, so `tb.run()` returns
- **Vector Sink** keeps everything; `.data()` hands it to NumPy
- **No throttle** — nothing here is tied to real time

Exercise 5.2 — controlling a running flowgraph

```
amplitude 1.00 -> measured power -0.00 dB
amplitude 0.50 -> measured power -6.02 dB
amplitude 0.10 -> measured power -20.00 dB
amplitude 0.01 -> measured power -40.00 dB
```

Halving the amplitude costs 6 dB — the table from Part 2, measured.

`tb.start()` returns at once; the flowgraph runs in background threads. `tb.stop()` then `tb.wait()`.

In between, your program can do anything: change parameters, read probes, serve a web page, track a satellite.

The kinds of blocks

Base class	Ratio	For	You write
<code>gr.sync_block</code>	1:1	maths, conversion, measurement	<code>work()</code>
<code>gr.decim_block</code>	N:1	averaging, downsampling	<code>work()</code>
<code>gr.interp_block</code>	1:N	upsampling, pulse shaping	<code>work()</code>
<code>gr.basic_block</code>	anything	packets, variable rates	<code>forecast()</code> , <code>general_work()</code>
<code>gr.hier_block2</code>	—	a reusable group of blocks	<code>connect()</code>

GNU Radio hands `work()` a chunk of input and a buffer for output; you return how many output samples you produced. The scheduler decides how big the chunks are — and it changes.

Exercise 5.3 — an Embedded Python Block

Drop a **Python Block** on the canvas, open it, and click *Open in Editor*. The code lives *inside* the .grc file — perfect for experiments and one-offs.

10_embedded_python_block.grc adds a DC offset to every sample, with a slider driving it.

Three conventions

- The class must be called `blk`
- Keyword arguments of `__init__` become **GRC parameters**
- An attribute with the same name gets a **callback** — which is how the slider reaches it while it runs

The template: imports and the class

```
import numpy as np
from gnuradio import gr
```

```
class blk(gr.sync_block):
    """DC Offset - adds a constant
       value to every sample"""
```

- numpy does the per-chunk arithmetic; GNU Radio hands you NumPy arrays
- `gr.sync_block`: one output sample per input sample
- The docstring becomes the block's *Documentation* tab

__init__: the block's signature

```
def __init__(self, dc_offset=0.5):
    gr.sync_block.__init__(
        self,
        name='DC Offset',
        in_sig=[np.float32],
        out_sig=[np.float32])
    self.dc_offset = dc_offset
```

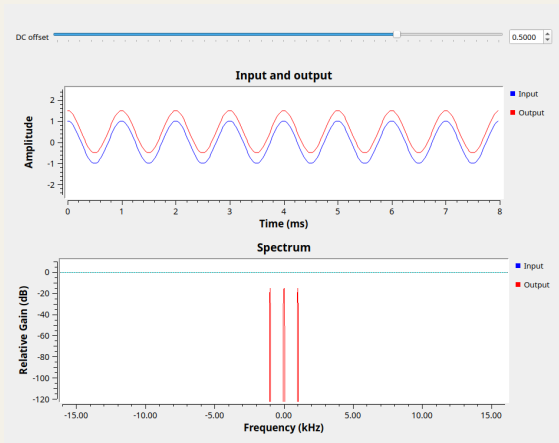
- name is for humans
- in_sig / out_sig are the port *signatures*: one port each, 32-bit float
- [np.complex64, np.float32] would be two input ports of different types
- The attribute makes the parameter changeable at runtime

work(): where the samples are

```
def work(self, input_items,
          output_items):
    output_items[0][:] = \
        input_items[0] + self.dc_offset
    return len(output_items[0])
```

- `input_items[0]` is a chunk of the first input port; `output_items[0]` is where its answer goes
- `[:]` writes *into* the buffer — do not rebind it
- **Return how many output samples you produced**
- The chunk size changes from call to call: never assume it

DC is zero frequency



Your turn: make it non-linear

```
output_items[0][:] = np.clip(input_items[0],  
                             -self.dc_offset,  
                             self.dc_offset)
```

Turn the slider down and watch a sine become a square — and harmonics appear that were never transmitted.

That is why an amplifier driven into clipping interferes with frequencies it was never meant to touch.

Exercise 5.4 — a decimator, and a test

```
class moving_average_ff(gr.decim_block):
    def work(self, input_items, output_items):
        out = output_items[0]
        groups = input_items[0][:len(out) * self.decimation]
        out[:] = groups.reshape(len(out),
                                self.decimation).mean(axis=1)

        return len(out)

def test_002_moving_average(self):
    src_data = (1, 1, 1, 1, 2, 4, 6, 8, 0, 0, 0, -4)
    expected = (1.0, 5.0, -1.0)
    ...
    self.assertFloatTuplesAlmostEqual(
        expected, snk.data(), 6)
```

Never assume the chunk size

`work()` is called with whatever is available. Always work from `len(output_items[0])`.

Vectorise

NumPy on the whole chunk is tens of times faster than a Python loop over samples.

Tags and messages

Stream tags ride *with* the samples: a key/value pair attached to one exact sample, travelling downstream in step with the data.

Hardware sources use them for `rx_time`, `rx_freq`, `rx_rate`.

Messages are asynchronous, sent between message ports, carrying PMTs. Use them for control, metadata and packets.

Tag Debug and **Message Debug** show you what is really going on.

06 —

Under the hood

Why it behaves the way it does, and
what to learn next.

How the scheduler runs your flowgraph

- Every block runs in **its own thread**
- Between every pair of blocks is a **circular buffer**
- The scheduler calls `work()` with whatever fits: samples available in, space free out

Back-pressure

A slow sink fills its input buffer; the block upstream finds no space and stalls; the stall travels back to the source.

Nothing overflows, and no block needs to know the overall rate — which is why *one* clock anywhere controls everything.

Telling the scheduler what you need

Mechanism	What it does
<code>forecast()</code>	inputs needed per output
<code>set_history(n)</code>	see the previous samples
<code>set_output_multiple(n)</code>	only multiples of n
<code>set_min_output_buffer()</code>	bigger buffers
Tag propagation policy	all-to-all, one-to-one, none

Writing blocks: pick the right base class

The scheduler can only help if you tell it the rate relationship. That is what the base class is for.

- **Sync** (1:1) — the common case
- **Decimation** (N:1) and **interpolation** (1:M) — fixed rate changes
- **Basic** (N:M) — everything else, and you do the book-keeping yourself

Sources and sinks are not special

A sync block with `in_sig=None` is a **source**; one with `out_sig=None` is a **sink** — then use `len(input_items[0])`, because `output_items` is empty.

Taxonomy and examples follow the GNU Radio wiki, *Types of Blocks*.

Sync block — 1:1

```
class my_sync_block(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(self,
                                name="my sync block",
                                in_sig=[np.float32, np.float32],
                                out_sig=[np.float32])

    def work(self, input_items, output_items):
        output_items[0][:] = (input_items[0]
                               + input_items[1])
        return len(output_items[0])
```

An adder: two input ports, one output.

- Every input and output vector has the *same* length
- That length is the C++ `noutput_items`
- Return the number produced

Decimation block — N:1

```
class my_decim_block(gr.decim_block):
    def __init__(self, decim_rate):
        gr.decim_block.__init__(self,
            name="my decim block",
            in_sig=[np.float32],
            out_sig=[np.float32],
            decim=decim_rate)
        self.decimation = decim_rate

    def work(self, input_items, output_items):
        output_items[0][:] = \
            input_items[0][0::self.decimation]
        return len(output_items[0])
```

Keep one sample in N — and remember Part 3: filter first, or you alias.

- decim tells the scheduler the ratio
- $\text{len(input)} = \text{len(output)} \times \text{decim}$, guaranteed

Interpolation block — 1:M

```
class my_interp_block(gr.interp_block):
    def __init__(self, interp_rate):
        gr.interp_block.__init__(self,
            name="my interp block",
            in_sig=[np.float32],
            out_sig=[np.float32],
            interp=interp_rate)
        self.interpolation = interp_rate

    def work(self, input_items, output_items):
        output_items[0][:] = np.repeat(
            input_items[0], self.interpolation)
        return len(output_items[0])
```

The mirror image: M outputs for every input.

- Real interpolators zero-stuff and then filter, to suppress the images

Basic block — N:M, you do the book-keeping

```
class my_basic_adder_block(gr.basic_block):
    def __init__(self):
        gr.basic_block.__init__(self,
            name="another_adder_block",
            in_sig=[np.float32, np.float32],
            out_sig=[np.float32])

    def general_work(self, input_items,
                    output_items):
        out = output_items[0]
        in0 = input_items[0][:len(out)]
        in1 = input_items[1][:len(out)]
        out[:] = in0 + in1
        self.consume(0, len(in0))
        self.consume(1, len(in1))
        return len(out)
```

- Override `general_work()`, not `work()`
- **You must** `consume()` what you used — the scheduler will not guess
- `consume_each(n)` is the shortcut for all ports
- Override `forecast()` to say how much input you need per output

Use this only when the others do not fit:
packet framing, variable-rate processing.

Making blocks permanent: OOT modules

```
gr_modtool newmod workshop
cd gr-workshop
gr_modtool add -t sync -l python \
    --argument-list "multiple=1.0" \
    --add-python-qa \
    --copyright "Your Name" my_multiply_ff
```

```
mkdir build && cd build
cmake .. && make
sudo make install && sudo ldconfig
```

Then *Reload Blocks* in GRC; in Python it is from `gnuradio import workshop`.

What you get

python/workshop/my_multiply_ff.py, a QA test beside it, and `grc/*.block.yml` describing it to GRC.

Ignore old tutorials

XML files and swig are GNU Radio 3.7. It is YAML now.

Where to go next

- The official guided tutorials on the wiki
- The examples in `share/gnuradio/examples/`
- Every block's *Documentation* tab
- **CGRAN** — hundreds of existing modules
- The Matrix chat, and GRCon talks on YouTube

Natural next projects

- FM RDS with `gr-rds`
- Aircraft with `gr-air-modes`
- Satellites with `gr-satellites`
- Digital modulation: synchronise, equalise, decode

Thank you



Questions?

Derek Kozel

